

A COURSE-BASED PROJECT
On
RAILWAY MANAGEMENT SYSTEM

Submitted in partial fulfillment of Operating Systems Lab

GRIET Lab On Board(G-LOB)

By

MOHAMMED ARHAN AHMED [25245A6703]

A.YASHWANT [24245A6704]



Department of Data science
GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous)
Bachupally, Kukatpally, Hyderabad, Telangana, India,500090
2023-2024



GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous)

Hyderabad-500090

CERTIFICATE

This is to certify that the GLOB entitled “**RAILWAY MANAGEMENT SYSTEM**” is submitted
Mohammed Arhan Ahmed (25245A6703), A. Yashwant (24245A6704) in partial fulfillment of
the award of degree in BACHELOR OF TECHNOLOGY in Computer Science and Engineering
(Data Science) during Academic year 2024-2025.

Internal Guide

- 1. Ms. Priyanka**
- 2. Ms. Tejaswini**

Table of Contents

Ch. No	Chapter Name	Pg. No.
1	Introduction	1
	1.1. Need for the Project	
	1.2. End Users of the Project	
	1.3 Objectives	
	1.4 Project Scope	
2	Requirement Analysis	5
3	Conceptual Database Design	6
4	Logical Database Design	9
5	SQL Code for Creation of Database	12
	5.1. Creation of Tables - DDL	12
	5.2. Referential Integrity Constraints - DDL	13
	5.3. Insert, Delete, and Update Data - DML	14
6	Retrieval of Data - DQL	15
7	Views	16
8	PL/SQL Code for retrieval and manipulations	17
	8.1. Cursors	17
	8.2. Triggers	17
	8.3. Functions/ Procedures	18
9	References/ Tools Used	19

1.Introduction

The **Railway Management System** is designed to handle and streamline the core operations of railway services, including ticket booking, passenger management, train details, and payment processing. The system maintains essential data related to trains, passengers, tickets, and payments to ensure efficient and secure ticket reservation and travel management.

Passengers can book tickets by selecting **train routes, travel dates, and destinations**. The booking information, along with passenger details and payment records, is stored in a structured manner to ensure accurate reservation tracking. Each booking is assigned a unique

PNR number, which links passenger information with the train and payment details.

The ER diagram represents the logical structure of the database, showing key entities such as **Train, Passenger, Train Ticket, and Payment** along with their attributes and relationships. It clearly defines how a passenger books a ticket, how payments are recorded, and how each ticket is associated with a specific train journey. This structured design ensures smooth data flow, reduces manual errors, and improves efficiency in railway operations.

1.1 Need for the Project

1. **Efficient Ticket Booking Process:**

Provides a streamlined system for passengers to search trains, check availability, and book tickets without long queues at railway stations, reducing manual work and saving time.

2. **Centralized Railway Information Storage:**

Maintains a unified database for trains, passengers, ticket bookings, and payments, eliminating manual record errors and ensuring accurate and consistent information across the system.

3. **Enhanced Passenger Experience:**

Offers a user-friendly platform where passengers can view train timings, routes, seat availability, and ticket status, making travel planning more convenient and hassle-free.

4. **Real-Time Booking and Availability Tracking:**

Tracks ticket reservations, cancellations, and train schedules in real time, helping passengers and railway staff access the latest information instantly.

5. **Secure Payment and Transaction Management:**

Facilitates smooth handling of ticket fare payments, payment modes, and transaction records, ensuring transparency and secure financial processing.

6. **Administrative Control and Railway Operations Support:**

Enables railway authorities to efficiently manage train schedules, passenger data, ticket records, and payments, improving service coordination and operational efficiency.

1.2. End Users of the Project

The Railway Management System provides passengers with a convenient and efficient platform to search for trains, check seat availability, book tickets, and view travel details. Passengers can easily access real-time train schedules, routes, ticket status, and fare information through a simple and user-friendly interface.

Railway staff and administrators benefit from streamlined operations, allowing them to manage train schedules, passenger information, ticket bookings, and payment records efficiently. The system ensures smooth handling of bookings, cancellations, and transaction details.

Through secure account handling and centralized data management, all users experience seamless communication, improved transparency, and reliable service. This system enhances the overall railway travel and management process by reducing manual effort, minimizing errors, and supporting efficient decision-making.

1.3. Objectives

The objectives of the Railway Management System project are:

1. Efficient Data Management:

Develop a structured database to store and manage information related to trains, passengers, tickets, routes, schedules, and payments in an organized manner.

2. Data Integrity:

Establish clear relationships between entities (e.g., passenger to ticket, ticket to train, payment to booking) to ensure data accuracy, consistency, and elimination of redundancy.

3. User Roles:

Provide separate functionalities for passengers (train search, ticket booking, payment) and railway administrators/staff (train schedule management, booking records, passenger data, and payment tracking).

4. Automation:

Automate key processes such as ticket booking, seat allocation, passenger record updates, cancellation handling, and payment recording to minimize manual effort and reduce errors.

5. Scalability:

Design the system to support future enhancements such as adding more trains, stations, services, and advanced features like real-time seat availability and multi-route booking.

1.4. Project Scope

The project focuses on developing a centralized database system for efficiently managing railway-related operations. It covers the management of trains, passengers, tickets, routes, and payment transactions. The system ensures data accuracy and reliability through well-defined relationships between entities such as Train, Passenger, Ticket, and Payment.

CRUD operations are supported for creating, updating, and deleting records related to trains, passenger details, bookings, and payment information. The system is designed to be scalable, enabling future expansion such as adding more trains, stations, routes, and advanced features as railway services grow.

Passengers can search for trains, view schedules, check seat availability, book tickets, and make secure payments, while railway administrators can monitor bookings, manage train

details, update schedules, and maintain passenger and transaction records. The system aims to provide a reliable, integrated, and user-friendly platform for both passengers and railway authorities, ensuring smooth operations and improved travel management.

2. Requirements Analysis

1. Database Structure:

The system requires a relational database consisting of multiple tables to manage information about trains, stations, passengers, tickets, routes, schedules, and payments. Proper relationships must be established to ensure data integrity, efficient storage, and quick data retrieval.

2. Train Management:

The system should store and manage detailed information about trains, including train number, train name, starting station, destination station, travel route, and schedule timings. Railway staff should be able to add, update, and delete train records as needed.

3. Passenger Information:

The system must maintain complete passenger details, such as passenger ID, name, age, contact information, and booking history. This supports proper passenger identification, reservation tracking, and travel management.

4. Ticket Booking and Authentication:

The system should allow passengers to search available trains, check seat availability, and book tickets by providing their details. Each booking must generate a unique PNR. The system must track ticket booking, cancellation status, and ensure secure handling of passenger information.

5. Payment and Transaction Handling:

The system should record payment details, including transaction ID, amount, payment mode, and payment time. This ensures transparency and allows verification of completed tickets and financial transactions.

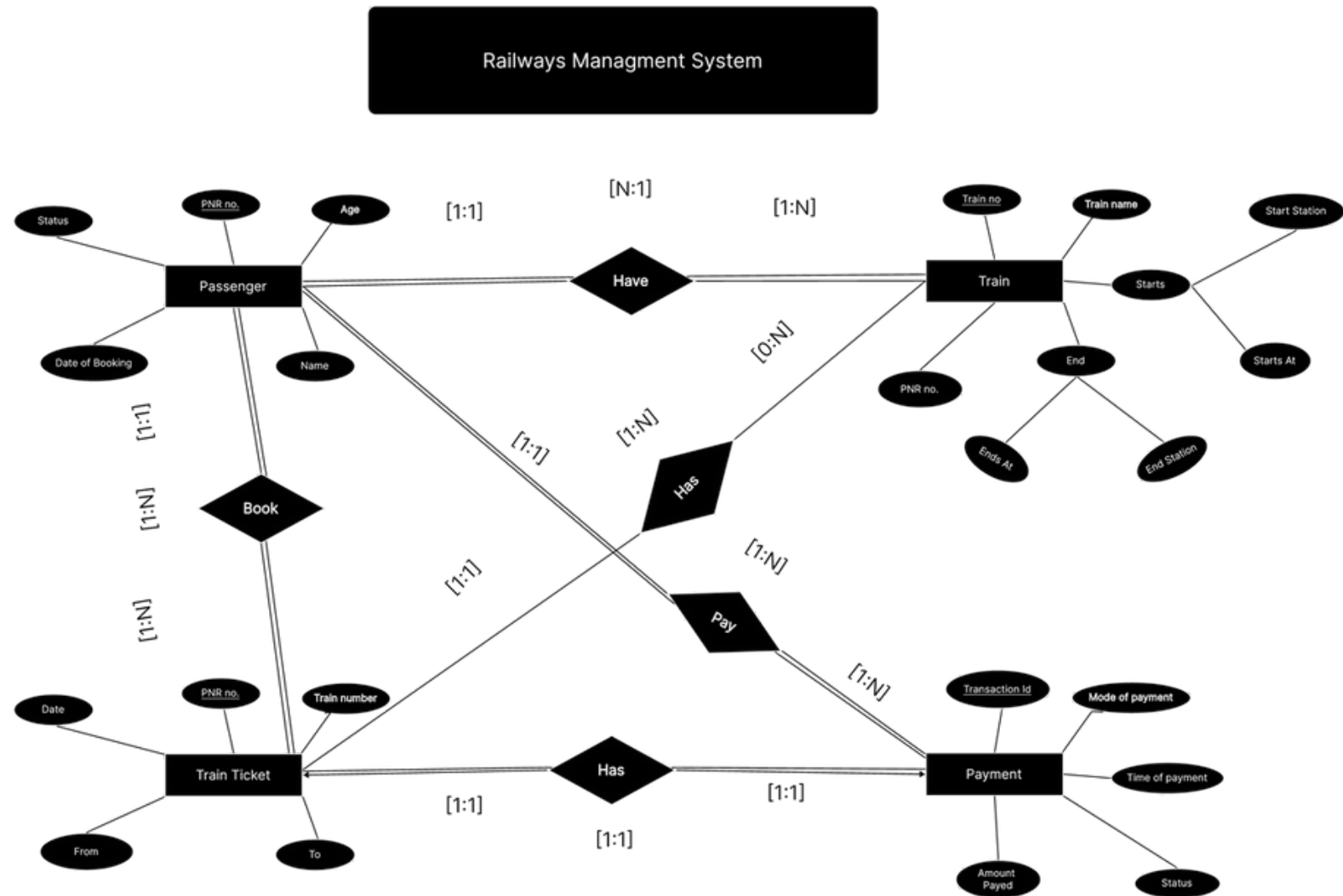
6. Booking & Journey Tracking:

The system must track which passengers have booked seats on which trains, ensuring mapping between passengers, bookings, and train details. This helps in seat allocation, travel record maintenance, and generating passenger lists if required.

3. Conceptual Database Design

The ER Model is created using Edraw

ER Diagram



Entities

- Train
- Passenger
- Train Ticket
- Payment

Attributes

1.Train

Attribute	Description
Train_No (Primary Key)	Unique train
Train_Name	number
Starts_At Ends_At	Name of the
Start _Station	train
End_Station	Starting
	station
	Ending
	station
	Starting location
	(station name)
	Ending location
	(station name)

2.Passenger

Attribut	Descriptio
e PNR_No (<i>Foreioftn Key – refers to Train Ticket</i>)	n Ticket reference for passenger
Passenger Id (Primary Key)	Unique Passenger ID
Nam e	Passenger name
Ag e	Passenger age
Date_Of_Booki ng	Booking date
Statu s	Ticket/Passenger status (Booked/Waiting/Cancelled etc.)

3.Train Ticket

Attribute	Description
PNR_No <i>(Primary Key)</i>	Unique
Train_No <i>(Foreign Key – refers to Train)</i>	booking ID
From To	Train number
Date	Boarding station
	Destination
	station Journey
	date

4. Payment

Attribute	Description
Transaction_ID <i>(Primary Key)</i>	Unique transaction
Mode_of_Payment	number UPI / Card /
Time_of_Payment	Cash etc.
Amount _Paid	Payment
PNR_No <i>(Foreign Key – refers to Train Ticket)</i>	timestamp Total
Status	fare amount
	Ticket payment mapping
	Payment status (Paid / Failed / Pending)

4. Logical Database Design

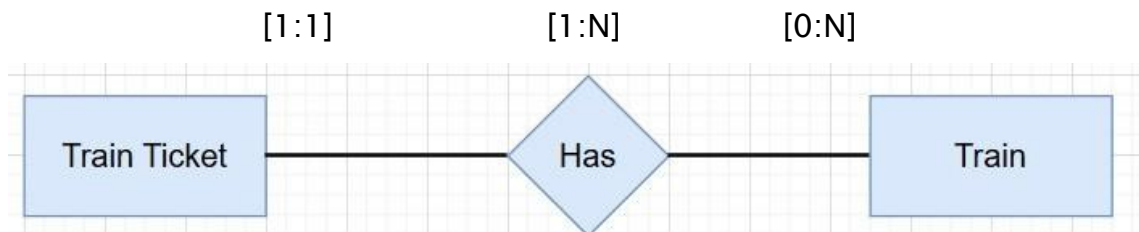
Identifying the strong entities

From the ER diagram Event, Organisers, Payments, Participants.

Identifying the One to Many relationships between the entities

1. Train to Train Ticket – One Train can have Multiple Tickets (1:N)

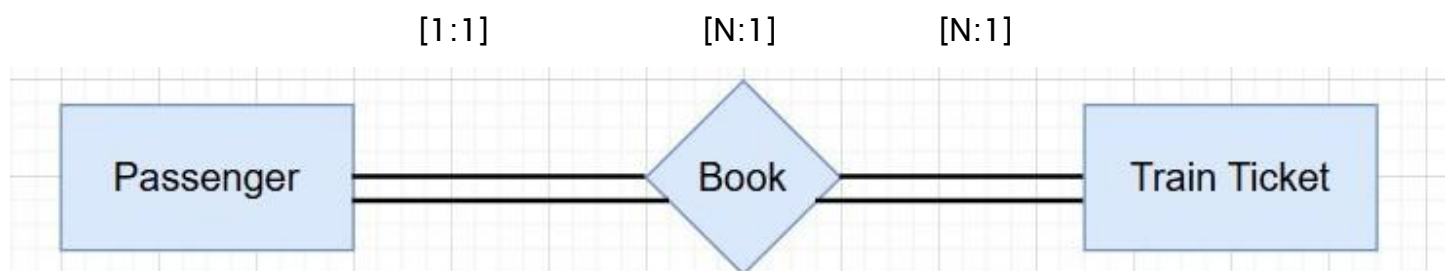
A single Train can be booked by many passengers, resulting in multiple Train Tickets. However, each Train Ticket is linked to only one Train.



2. Passenger to Train Ticket – One Passenger has One Ticket (1:1)

Each Passenger is associated with only one Train Ticket, and each Train Ticket corresponds to one Passenger.

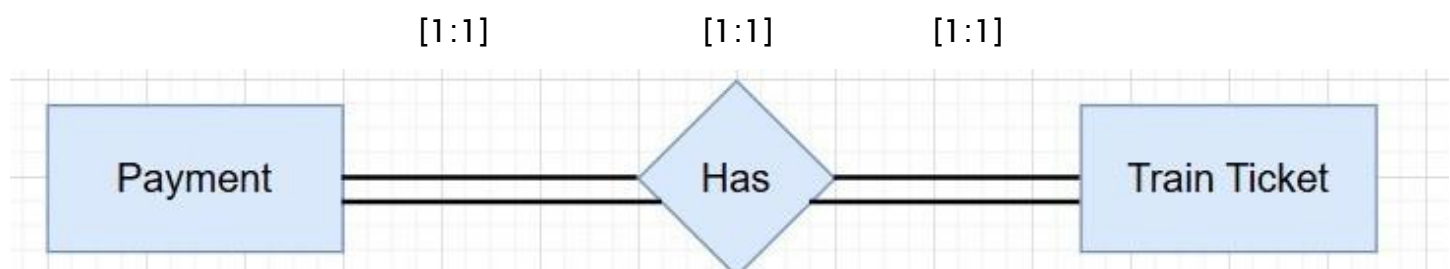
This ensures unique mapping of passenger details for each booking (PNR).



3. Train Ticket to Payment – One Ticket has One Payment (1:1)

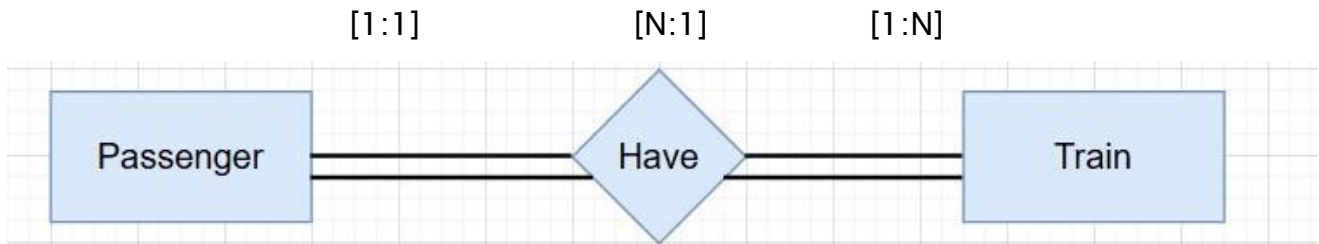
Every Train Ticket has a corresponding Payment record for the fare, and each Payment is linked to one specific Train Ticket.

This ensures secure financial tracking for every booking.



4. Train to Passenger – Indirect Relationship through Train Ticket (1:N)

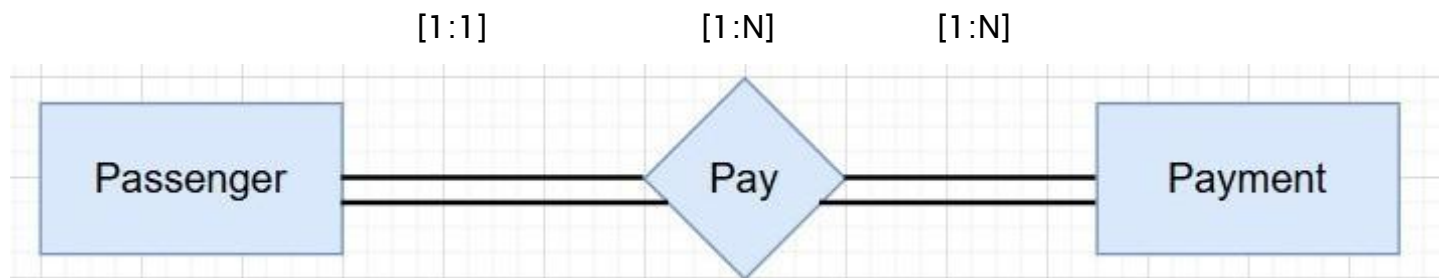
A Train can carry multiple passengers, but this connection occurs via the Train Ticket entity. Thus, **Train** → **Ticket (1:N)** and **Ticket** → **Passenger (1:1)** implies **Train** → **Passenger (1:N)**



5. Passenger to Payment – One Passenger has One Payment (1:1)

A Passenger makes one payment for their booking, and each Payment entry belongs to only one Passenger.

This relationship is implemented indirectly through the Train Ticket entity, where PNR acts as the linking key.



3. Schema Design

The schema for the Railway Management System is created to ensure proper normalization, avoiding redundancy while maintaining referential integrity among entities.

Primary Keys:

- Train: TrainID
- Passenger: PassengerID
- Ticket: TicketID
- Payment: PaymentID
- Employee: EmployeeID
- Station: StationID

Foreign Keys:

- Passenger.TrainID → references Train(TrainID)
- Ticket.TrainID → references Train(TrainID)
- Ticket.PassengerID → references Passenger(PassengerID)
- Ticket.PaymentID → references Payment(PaymentID)
- Train.Start_Station and Train.End_Station → reference Station(StationID)
- Train.EmployeeID → references Employee(EmployeeID)

4. Constraints on the Relations

1. Primary Keys

- Each entity has a unique identifier to prevent duplicate records and ensure distinct identification of each row.

2. Foreign Keys

Maintain relational consistency between dependent tables.

Example: A ticket cannot exist without a valid passenger or train.

3. Unique Constraints

Applied to fields such as Train_No, PNR_No, and Transaction_ID to prevent duplication of important identifiers.

4. NOT NULL Constraints

Essential attributes like Passenger.Name, Train.Train_Name, Payment.Amount_Paid, and Ticket.Date are made mandatory to prevent incomplete records.

5. Referential Integrity

Ensures that all foreign key references point to valid existing records.

A payment cannot exist without a passenger.

A ticket cannot reference a non-existing train or passenger. A train must belong to valid start and end stations.

6. Data Deletion Constraints

Cascade deletion rules are defined carefully:

- If a passenger is deleted, all related tickets and payments are also removed.
- If a train is deleted, associated tickets become invalid unless handled through proper ON DELETE logic.

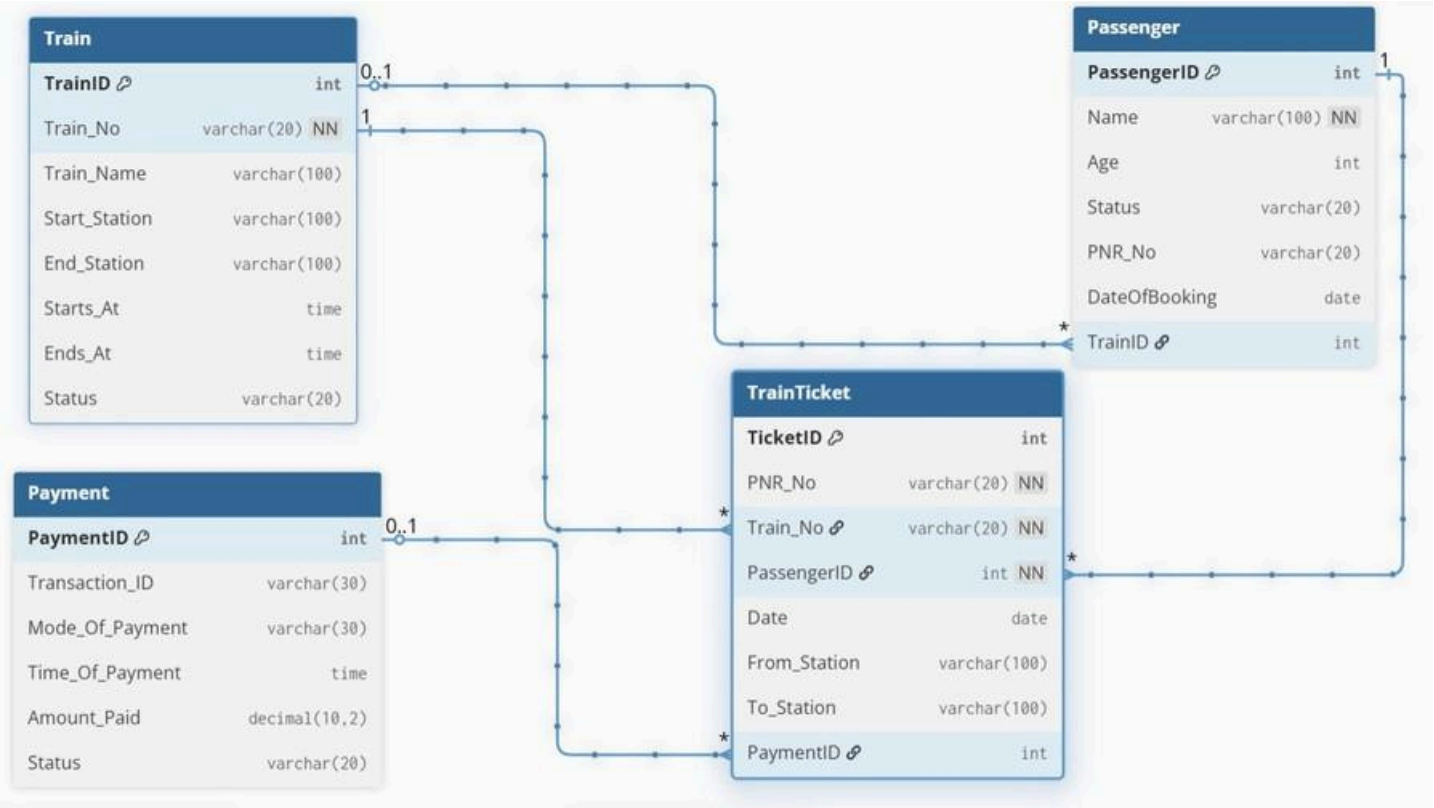
-
-

7. Implicit Integrity

Natural constraints emerge from the relationships themselves — for

- instance: A ticket always implies the existence of both a passenger and a train.
 - Payments are inherently tied to valid ticket records.
-

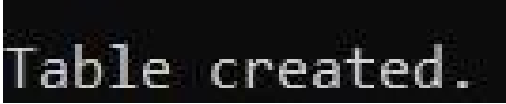
Schema Design



5.1. SQL Code for Creation of tables

Passenger table

```
CREATE TABLE Passenger (  
  PassengerID INT PRIMARY KEY,  
  Name VARCHAR(100) NOT NULL,  
  Age INT,  
  Status VARCHAR(20),  
  
  PNR_No VARCHAR(20) UNIQUE,  
  DateOfBooking DATE  
);
```

A terminal window with a black background and white text displaying the message "Table created." in a monospaced font.

Train table

```
CREATE TABLE Train (  
  TrainID INT PRIMARY KEY,  
  Train_No VARCHAR(20) UNIQUE NOT NULL,  
  Train_Name VARCHAR(100),  
  Start_Station VARCHAR(100),  
  End_Station VARCHAR(100),  
  Starts_At DATE,  
  Ends_At DATE,  
  Status VARCHAR(20)  
);
```

A terminal window with a black background and white text displaying the message "Table created." in a monospaced font.

Train Ticket table

```
CREATE TABLE TrainTicket (  
  TicketID INT PRIMARY KEY,  
  PNR_No VARCHAR(20) UNIQUE NOT NULL,
```

```
Train_No VARCHAR(20) NOT NULL,  
PassengerID INT NOT NULL,  
"Date" DATE,  
From_Station VARCHAR(100),  
To_Station VARCHAR(100),  
PaymentID INT  
);
```

Table created.

Payment Table

```
CREATE TABLE Payment (  
PaymentID INT PRIMARY KEY,  
Transaction_ID VARCHAR(30) UNIQUE,  
Mode_Of_Payment VARCHAR(30),  
Time_Of_Payment DATE,  
Amount_Paid DECIMAL(10,2),  
Status VARCHAR(20)  
);
```

Table created.

5.2. Referential Integrity Constraints

– DDL Creating foreign key in

Passenger table

```
ALTER TABLE Passenger ADD (TrainID  
NUMBER);  
ALTER TABLE Passenger  
  
ADD CONSTRAINT fk_train FOREIGN KEY  
(TrainID) REFERENCES Train(TrainID);
```

Table altered.

Creating foreign key in TrainTicket table

```
ALTER TABLE TrainTicket  
  
ADD CONSTRAINT fk_train_ticket FOREIGN KEY (Train_No) REFERENCES Train(Train_No),  
  
ADD CONSTRAINT fk_passenger_ticket FOREIGN KEY (PassengerID) REFERENCES  
Passenger(PassengerID),  
  
ADD CONSTRAINT fk_payment_ticket FOREIGN KEY (PaymentID) REFERENCES  
Payment(PaymentID);
```

Table altered.

5.3. Insert, Delete, and Update Data – DML Inserting data into Train Table

```
INSERT INTO Train (Train_No, Train_Name, Start_Station, End_Station, Starts_At, Ends_At, Status) VALUES ('12345', 'Express A', 'City A', 'City B', '09:00:00', '18:00:00', 'Active');
```

```
1 row created.
```

Inserting data into Passenger Table

```
INSERT INTO Passenger (PassengerID, Name, Age, Status, PNR_No, DateOfBooking, TrainID) VALUES (1, 'Rahul Kumar', 27, 'Confirmed', 'PNR1001', TO_DATE('2025-02-01', 'YYYY-MM-DD'), 1);
```

```
1 row created.
```

Inserting data into Payment Table

```
INSERT INTO Payment (PaymentID, Transaction_ID, Mode_Of_Payment, Time_Of_Payment, Amount_Paid, Status) VALUES (1, 'TXN9087', 'Online', TO_DATE('2025-02-05 10:30:00', 'YYYY-MM-DD HH24:MI:SS'), 1500, 'Paid');
```

```
1 row created.
```

Inserting data into TrainTicket Table

```
INSERT INTO TrainTicket (TicketID, PNR_No, Train_No, PassengerID, "Date", From_Station, To_Station, PaymentID) VALUES (1, 'PNR1001', '12345', 1, TO_DATE('2025-02-05', 'YYYY-MM-DD'), 'City A', 'City B', 1);
```

```
1 row created.
```

Update status of passenger

```
UPDATE Passenger SET Status='Waiting' WHERE PassengerID=1;
```

```
1 row updated.
```

Delete Data from TrainTicket

```
DELETE FROM TrainTicket WHERE TicketID=3;
```

```
0 rows deleted.
```

6. Retrieval of Data – DQL

Retrieve Passenger booking details

```
SELECT p.Name, t.Train_Name, tk."Date", tk.From_Station, tk.To_Station
FROM Passenger p
JOIN TrainTicket tk ON p.PassengerID = tk.PassengerID
JOIN Train t ON tk.Train_No = t.Train_No;
```

NAME	TRAIN_NAME	Date
Rahul Kumar	Express A	05-FEB-25
City A		
City B		

Show payments done by passengers

```
SELECT p.Name, pay.Amount_Paid, pay.Mode_Of_Payment
FROM Passenger p
JOIN TrainTicket tk ON p.PassengerID = tk.PassengerID
JOIN Payment pay ON tk.PaymentID = pay.PaymentID;
```

NAME	AMOUNT_PAID	MODE_OF_PAYMENT
Rahul Kumar	1500	Online

List of all active Trains

```
SELECT Train_Name, Train_No FROM Train WHERE Status='Active';
```

TRAIN_NAME	TRAIN_NO
Express A	12345

7.Views

Create a view to display passenger details along with train name, travel date, and amount paid.

```
CREATE VIEW View_PassengerDetails AS
SELECT p.Name, p.Age, p.PNR_No, t.Train_Name, tk."Date", pay.Amount_Paid
FROM Passenger p
JOIN TrainTicket tk ON p.PassengerID = tk.PassengerID
JOIN Train t ON tk.Train_No = t.Train_No
JOIN Payment pay ON tk.PaymentID = pay.PaymentID;
```

```
View created.
```

To display the created view

```
SELECT * FROM View_PassengerDetails;
```

NAME	AGE	PNR_NO
TRAIN_NAME	Date	AMOUNT_PAID
Rahul Kumar	27	PNR1001
Express A	05-FEB-25	1500

8. PL/SQL Code for Retrieval and Manipulations

8.1. Cursor

Display all the train numbers and names stored in the Train table

```
DECLARE
CURSOR train_cursor IS
SELECT Train_Name, Train_No FROM Train;
v_name Train.Train_Name%TYPE;
v_no Train.Train_No%TYPE;
BEGIN
OPEN train_cursor;
LOOP
    FETCH train_cursor INTO v_name, v_no;
    EXIT WHEN train_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_name || ' - ' || v_no);
END LOOP;
CLOSE train_cursor;

END;
/
```

```
PL/SQL procedure successfully completed.
```

8.2. Triggers

Create a trigger to validate payment entries before storing in the database

```
CREATE OR REPLACE TRIGGER check_amount
BEFORE INSERT ON Payment
FOR EACH ROW
BEGIN
    IF :NEW.Amount_Paid <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Invalid Amount!');
    END IF;
END;

/
```

```
Trigger created.
```

8.3. Functions / Procedures

Function to get Passenger count

```
CREATE OR REPLACE FUNCTION TotalPassengers  
RETURN NUMBER IS  
  cnt NUMBER;  
BEGIN  
  SELECT COUNT(*) INTO cnt FROM Passenger;  
  RETURN cnt;  
END;  
/
```

Function created.

Procedure to update train status

```
CREATE OR REPLACE PROCEDURE UpdateTrainStatus(  
  trainNo IN VARCHAR2  
)  
IS  
BEGIN  
  UPDATE Train  
    SET Status = 'Inactive'  
  WHERE Train_No = trainNo;  
END;  
/
```

Procedure created.

9. References

1. **Edraw.ai** (To create ER Diagram)
2. **“Database System Concepts”** – Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, McGraw Hill Education, 7th Edition.
3. **“Database Management Systems”** – Raghu Ramakrishnan and Johannes Gehrke, Tata McGraw Hill, 3rd Edition.
4. **“Fundamentals of Database Systems”** – Ramez Elmasri and Shamkant B. Navathe, Pearson Education.
5. **“Introduction to Database Systems”** – C.J. Date, Pearson Education.
6. **“SQL & PL/SQL for Oracle 19c – Black Book”** – Dr. P.S. Deshpande, Dreamtech Press.
7. **“The Complete Reference: SQL”** – James R. Groff, Paul N. Weinberg, and Andy Oppel, McGraw Hill.
8. **“Database Systems: Design, Implementation, and Management”** – Rob & Coronel, Cengage Learning, 12th Edition.
9. **MySQL / Oracle Official Documentation** – Database engine syntax and PL/SQL reference guides.